

Recursively Placing the Image Within the Image

Once the base image has been resized, we can place the resized image over the base image. But the pixels from the resized image should be placed over only magenta pixels in the base image. The resized image will be placed such that the top-left corner of the resized image is at the top-left corner of the magenta area:

Python

```
# Replace the magenta pixels with the smaller, resized image:
for x in range(magentaLeft, magentaRight + 1):
    for y in range(magentaTop, magentaBottom + 1):
        if baseImage.getpixel((x, y)) == magentaColor:
            pix = resizedImage.getpixel((x - magentaLeft, y - magentaTop))
            baseImage.putpixel((x, y), pix)
```

Two nested `for` loops iterate over every pixel in the magenta area. Remember that the magenta area does not have to be a perfect rectangle, so we check whether the pixel at the current coordinates is magenta. If so, we get the pixel color from the corresponding coordinates in the resized image and place it on the base image. After the two nested `for` loops have finished looping, the magenta pixels in the base image will have been replaced by pixels from the resized image.

However, the resized image itself could have magenta pixels, and if so, these will now become part of the base image, as in the top-right image of [figure 14-2](#). We'll need to pass the modified base image to a recursive `makeDroste()` call:

Python

```
# RECURSIVE CASE:
return makeDroste(baseImage, stopAfter - 1)
```

This line is the recursive call in our recursive algorithm, and it's the last line of code in the `makeDroste()` function. This recursion handles the new